

CS707 – Generative Models for NLP and Multimodal Reasoning

2026-27 Term 1

Week 3: Transformers and Sequence Modeling

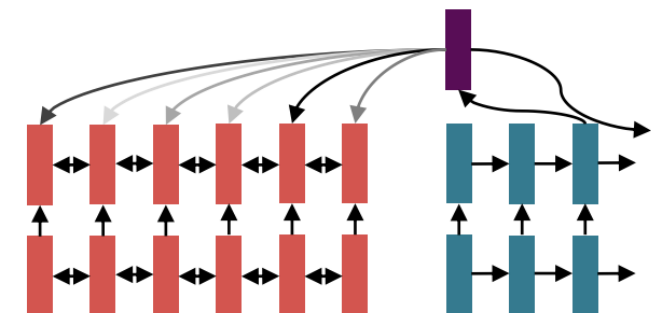
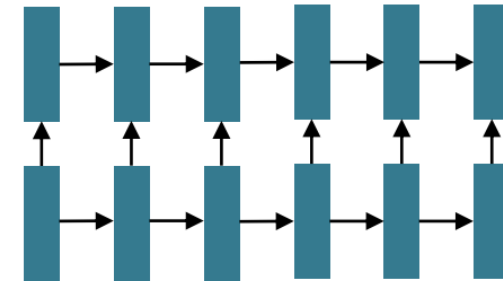
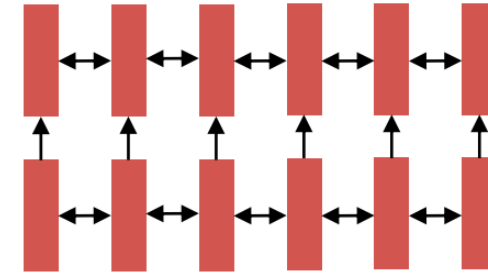
Dr. Wei Gao
Office: SCIS-1/4057
Email: weigao@smu.edu.sg
Tel.: 68261369

Outline

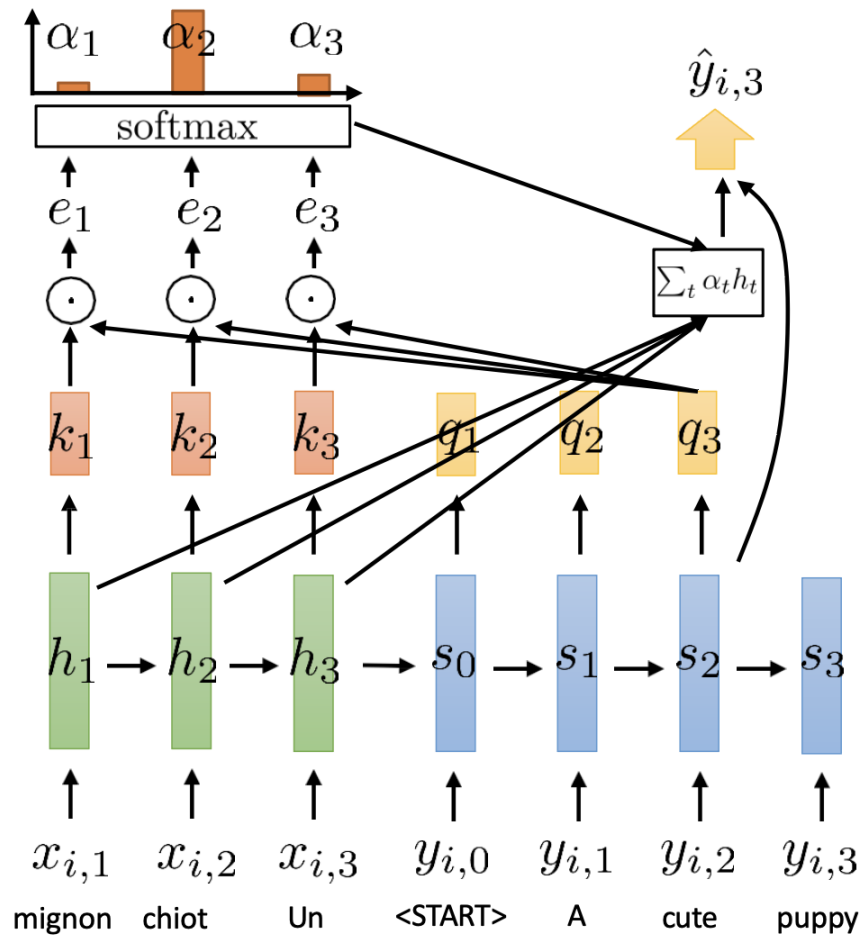
- **Self-Attention**
- **From Self-Attention to Transformers**
- **Transformer Encoder and Decoder**
- **How to Fix**

Recurrent Models for (most) NLP

- The de facto strategy in NLP is to encode sentences with a bidirectional LSTM: (e.g., the source sentence in a translation)
- Define your output (parse, sentence, summary) as a sequence, and use an LSTM to generate it.
- Use attention to allow flexible access to memory



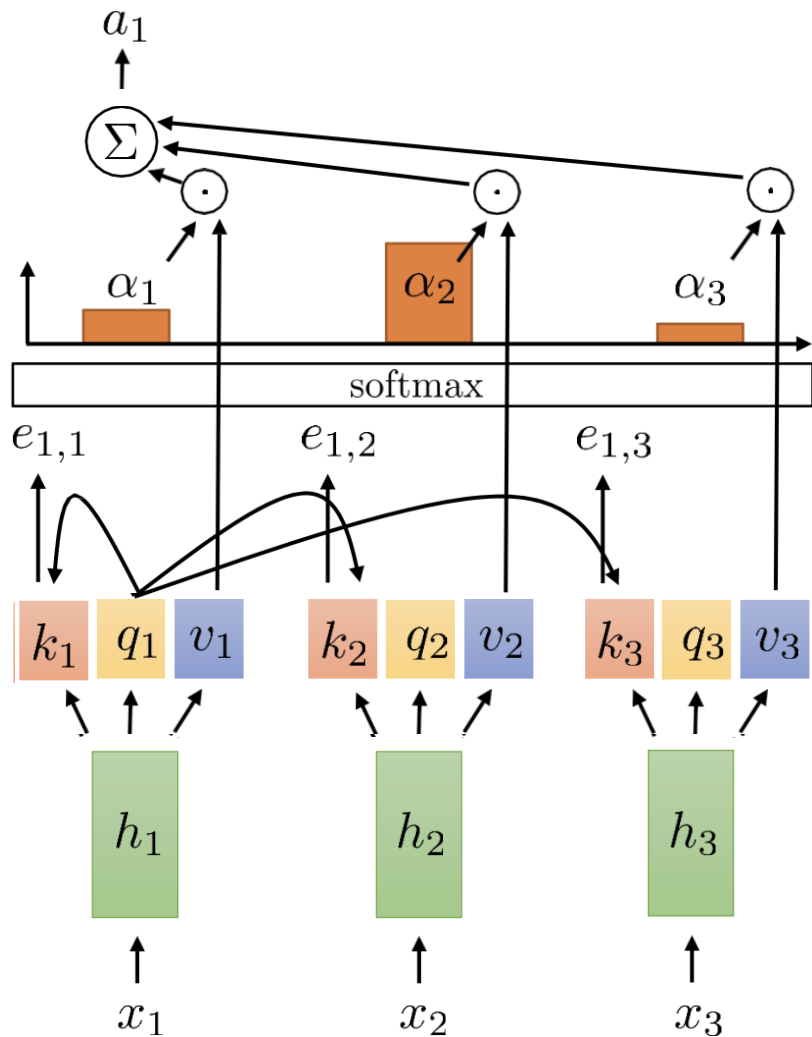
Attention



- If we have **attention**, do we even need recurrent connections?
- Can we transform our RNN into a **purely attention-based model**?
- Attention can access all time steps simultaneously, potentially doing everything that recurrence can, and even more. However, this approach presents some challenges:

Pure attention has no inherent notion of taken order or temporal position.

Self-Attention



$$a_l = \sum_t \alpha_{l,t} \cdot v_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g., $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

This is not a recurrent model! But still weight sharing:

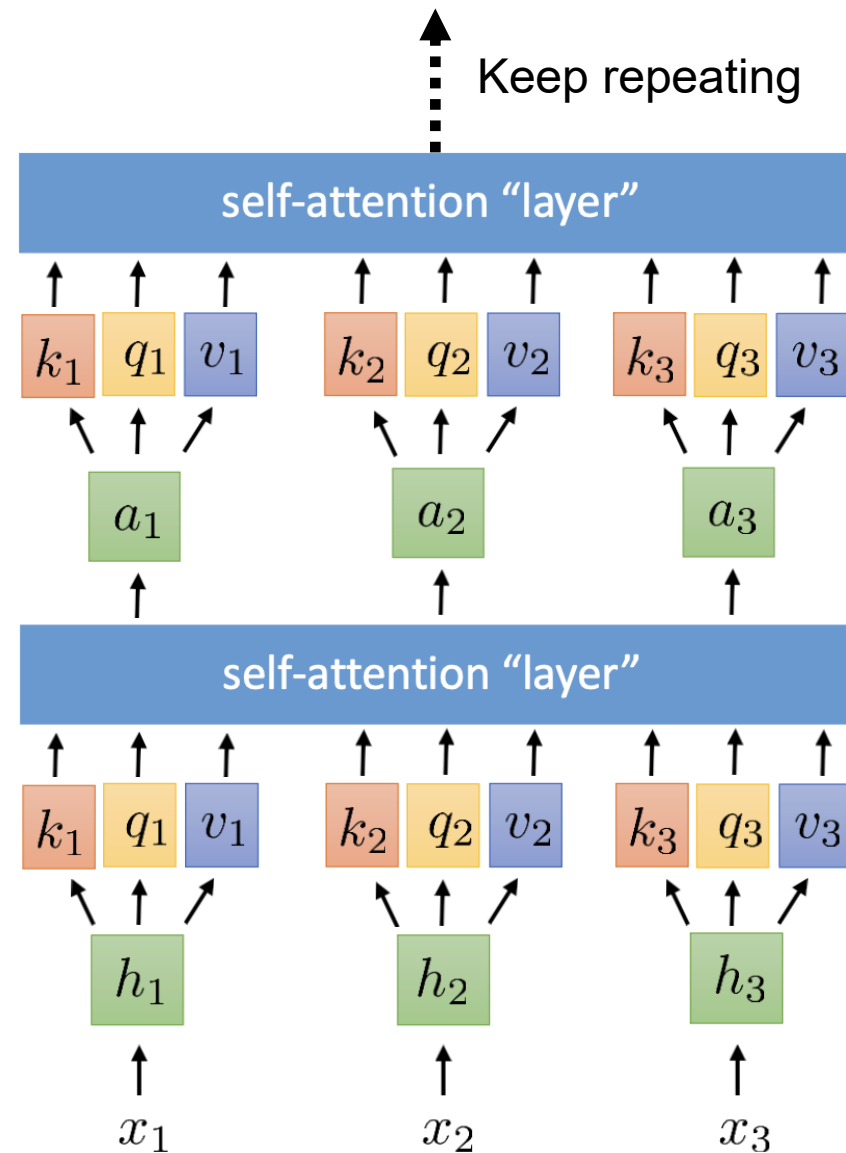
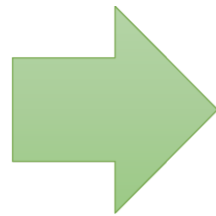
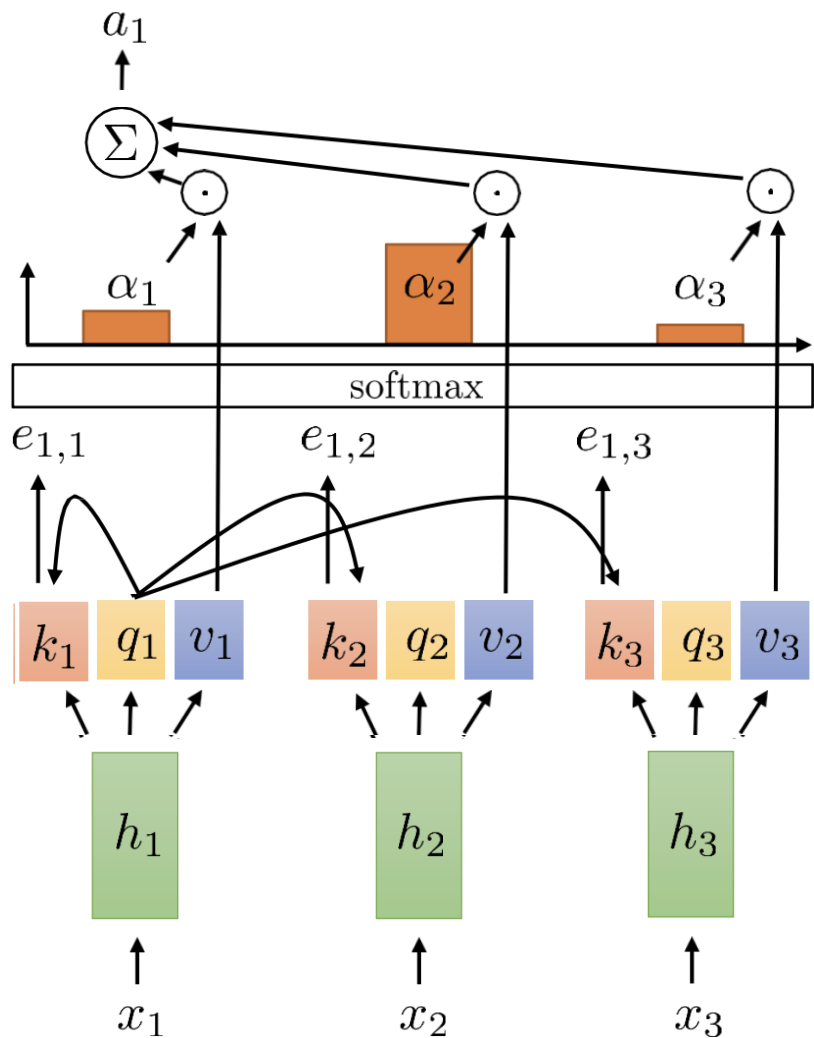
$$h_t = \sigma(Wx_t + b)$$



shared weights at all time steps

(or any other nonlinear function)

Self-Attention



Outline

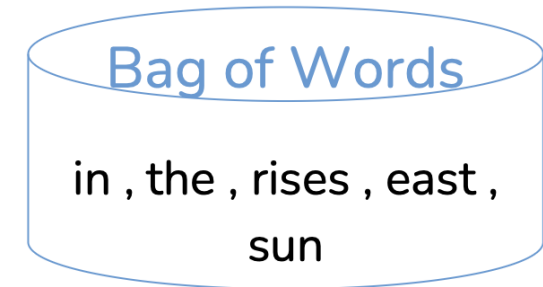
- Self-Attention
- **From Self-Attention to Transformers**
- **Transformer Encoder and Decoder**
- **How to Fix**

From Self-Attention to Transformers

- We will talk about a class of models for processing sequences that does not use recurrent connections but instead relies entirely on attention and will build up towards a class of models called **Transformers**.
- To address a few key limitations, we need to add certain elements:
 1. **Positional encoding**: Addresses lack of sequence information
 2. **Multi-headed attention**: Allows multiple attention patterns to be learned in parallel
 3. **Adding nonlinearities**: So far, each successive layer is linear in the previous one
 4. **Masked decoding**: How to prevent attention lookups into the future?

Positional Encoding - Motivation

- **Problem:** Self-attention processes all the elements of a sequence in parallel without any regard for their order.
 - Example : the sun rises in the east
 - Permuted version : rises in the sun the east
the east rises in the sun
- Self-attention is permutation invariant.
- In natural language, it is important to take into account the order of words in a sentence.
- **Solution:** Explicitly add positional information to indicate where a word appears in a sequence

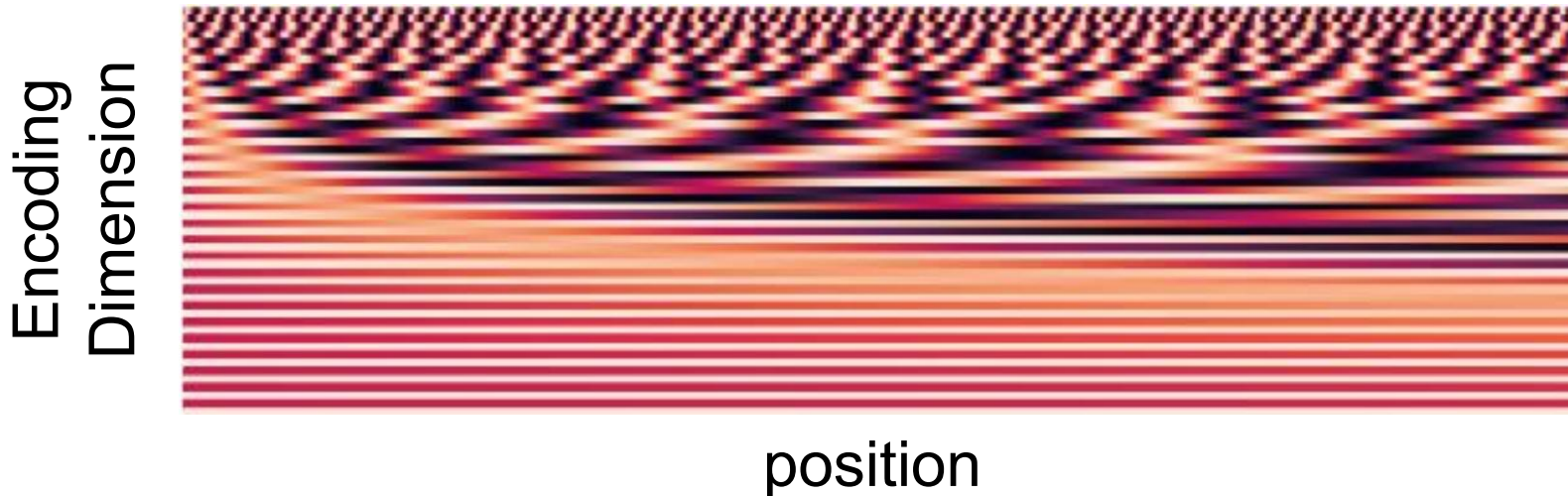


Sinusoidal Positional Encoding

- Helps it determine the position of each word (absolute positional information), or the distance between different words in the sequence (relative positional information)
- The frequency decreases along the encoding dimension.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



- **Pos**: the **position index** of the token in the sequence
- **i**: the sine/cosine **frequency pair index** inside the embedding vector.

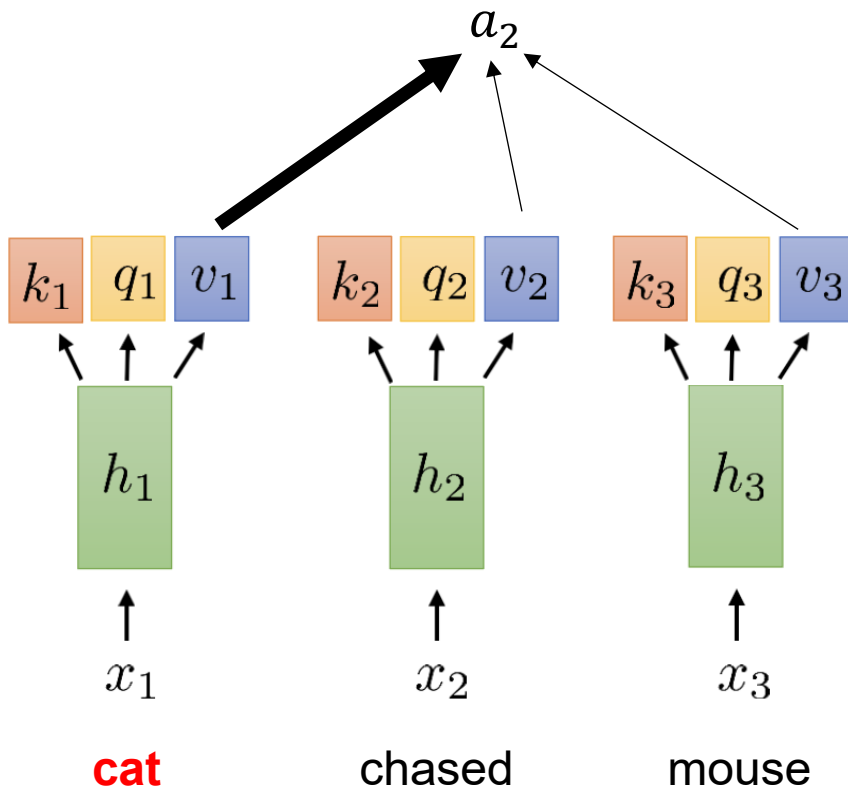
$i = 0, 1, \dots, \frac{d_{\text{model}}}{2} - 1$ correspond to different “frequencies” of sine/cosine waves.

From Self-Attention to Transformers

- We will talk about a class of models for processing sequences that does not use recurrent connections but instead relies entirely on attention and will build up towards a class of models called **Transformers**.
- To address a few key limitations, we need to add certain elements:
 1. **Positional encoding**: Addresses lack of sequence information
 2. **Multi-headed attention**: Allows querying multiple positions at each layer
 3. **Adding nonlinearities**: So far, each successive layer is linear in the previous one
 4. **Masked decoding**: How to prevent attention lookups into the future?

Multi-Head Attention

Given that we're fully depending on attention, it could be beneficial to capture **multiple different relationships** among sequence positions **simultaneously**.



$$a_l = \sum_t \alpha_{l,t} v_t$$

Due to the **softmax** function, this will be heavily influenced by a single value (WHY?)

$$e_{l,t} = q_l \cdot k_t$$

It's challenging to clearly specify that you want two distinct elements, like the subject ("cat") and object ("mouse") in a sentence, to be captured simultaneously.

Multi-Head Attention

Solution: Use multiple keys, queries, and values for each time step

Full attention vector formed by concatenation:

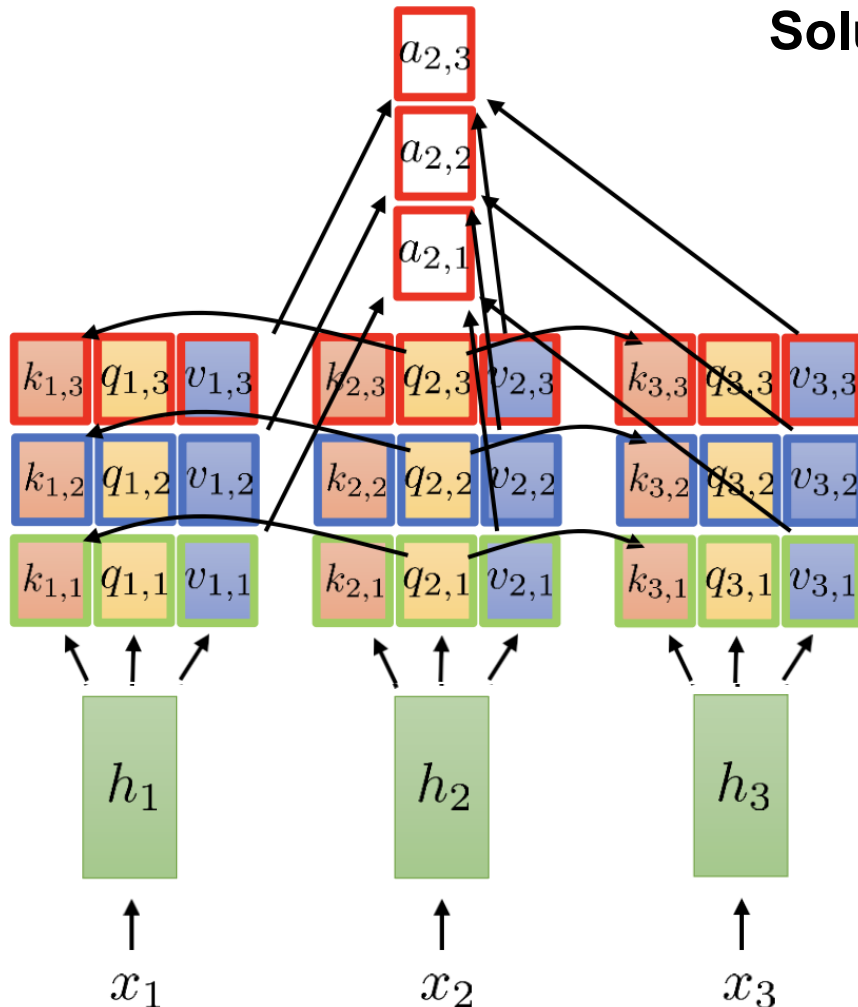
$$a_2 = \begin{bmatrix} a_{2,1} \\ a_{2,2} \\ a_{2,3} \end{bmatrix}$$

Compute weights **independently** for each head

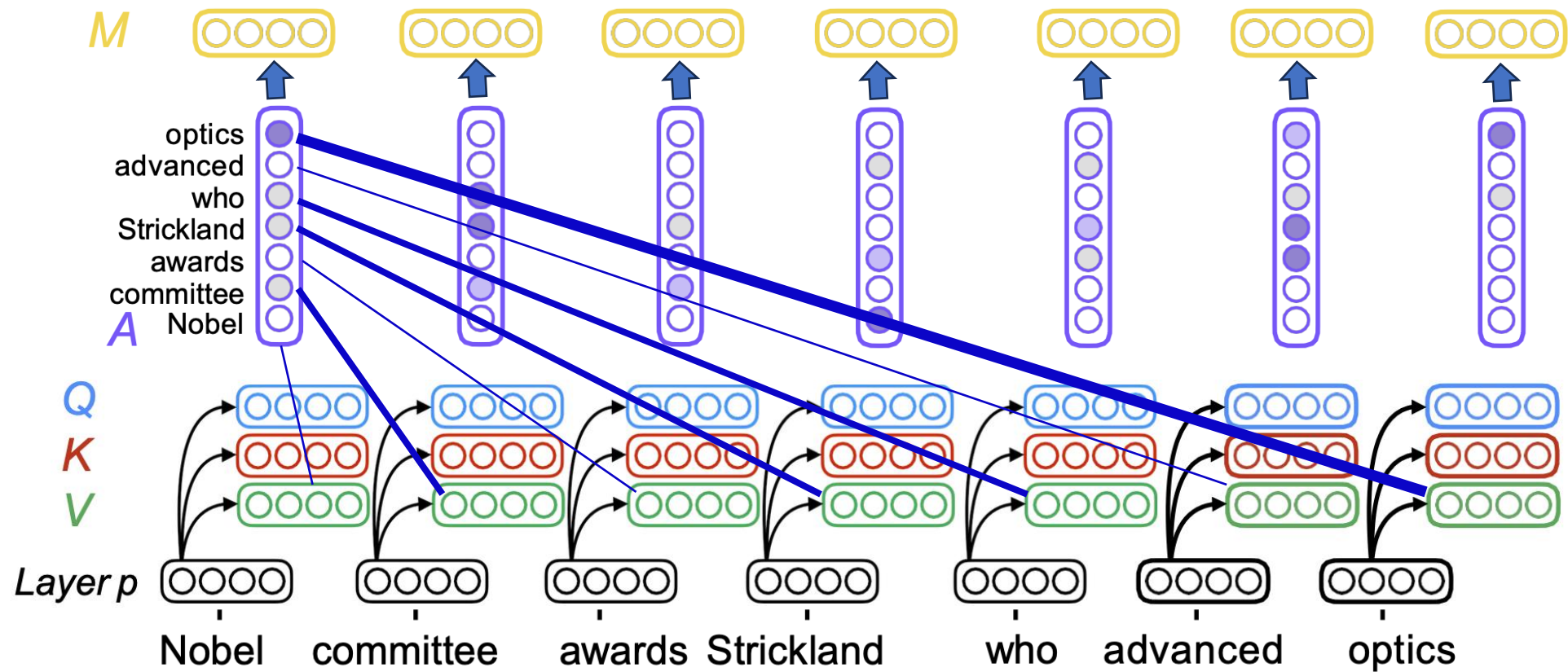
$$e_{l,t,i} = q_{l,i} \cdot k_{t,i}$$

$$\alpha_{l,t,i} = \exp(e_{l,t,i}) / \sum_{t'} \exp(e_{l,t',i})$$

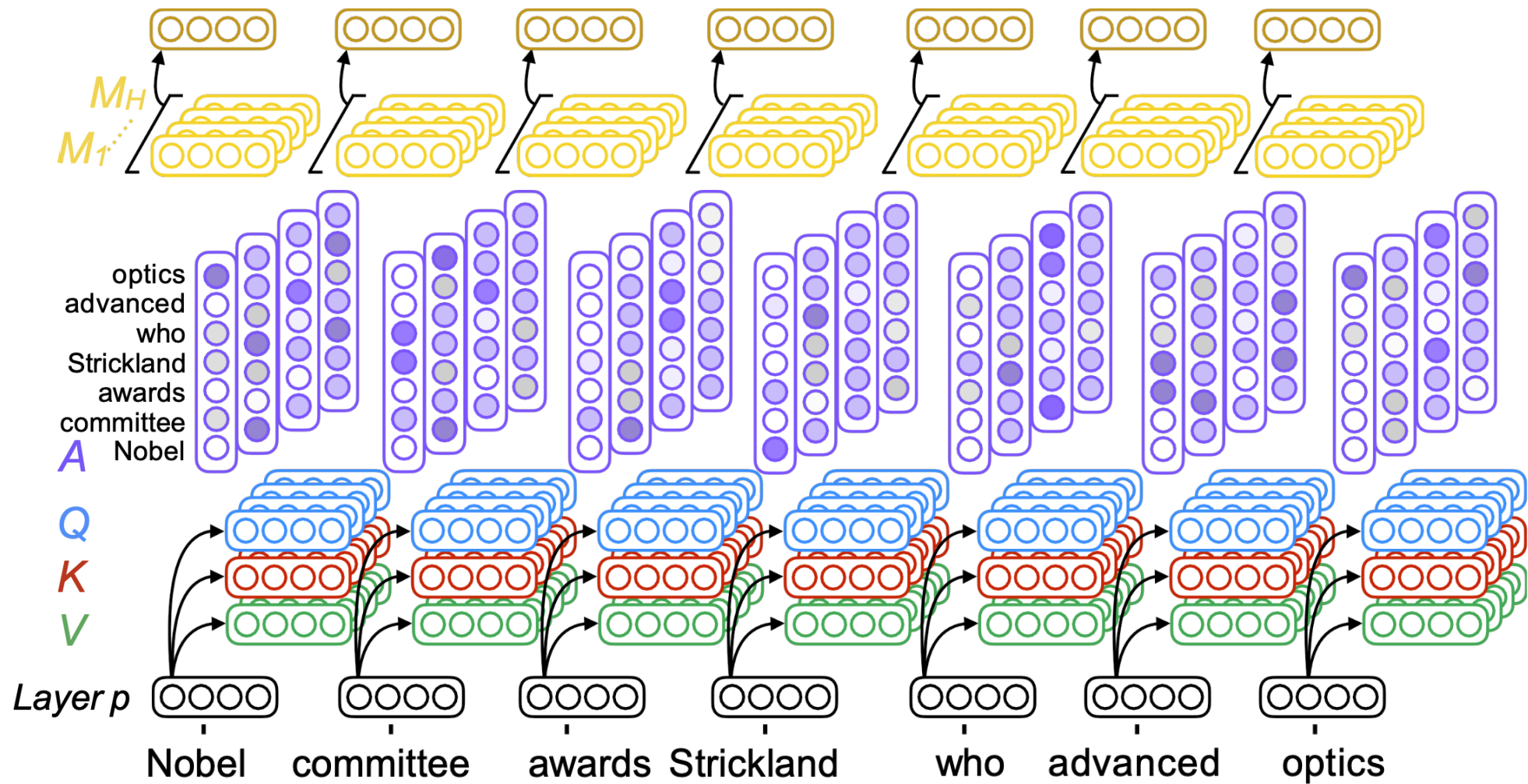
$$a_{l,i} = \sum_t \alpha_{l,t,i} v_{t,i}$$



Self-Attention (In Encoder)



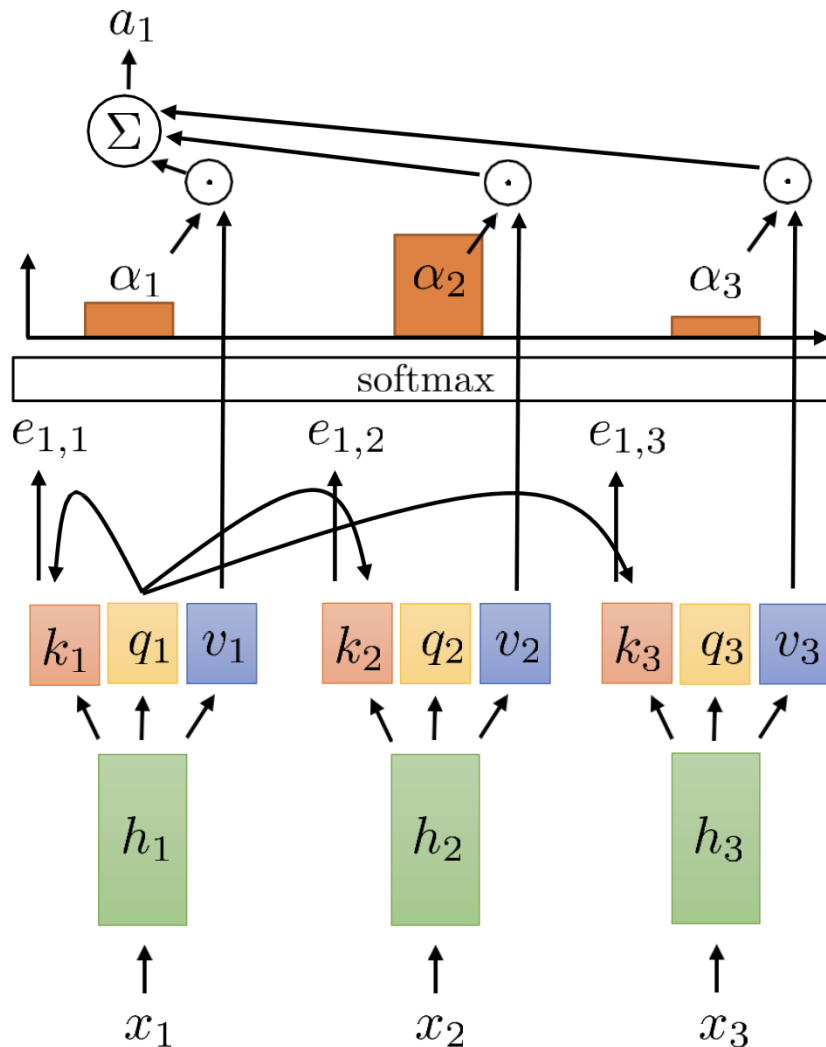
Multi-Head Self-Attention



From Self-Attention to Transformers

- We will talk about a class of models for processing sequences that does not use recurrent connections but instead relies entirely on attention and will build up towards a class of models called **Transformers**.
- To address a few key limitations, we need to add certain elements:
 1. **Positional encoding**: Addresses lack of sequence information
 2. **Multi-headed attention**: Allows querying multiple positions at each layer
 3. **Adding nonlinearities**: So far, each successive layer is linear in the previous one
 4. **Masked decoding**: How to prevent attention lookups into the future?

Self-Attention Is “Linear”



$$k_t = W_k h_t, \quad q_t = W_q h_t, \quad v_t = W_v h_t$$

$$e_{l,t} = q_l \cdot k_t$$

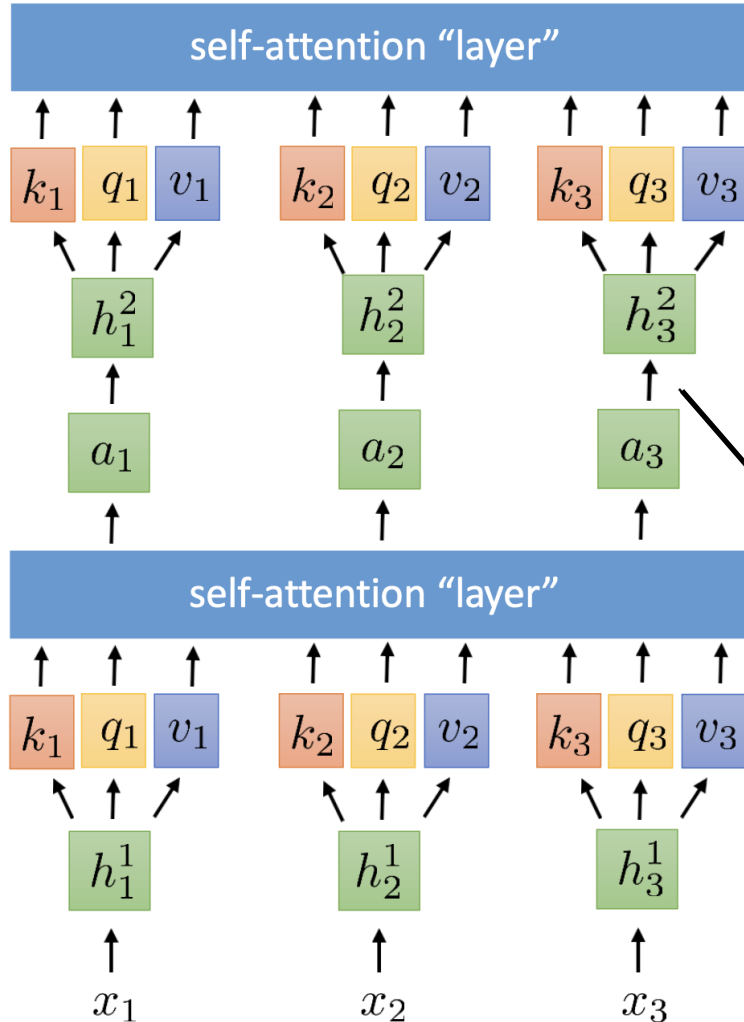
$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

$$a_l = \sum_t \alpha_{l,t} v_t = \sum_t \alpha_{l,t} W_v h_t = W_v \sum_t \alpha_{l,t} h_t$$

linear transformation non-linear weights

Problem: Every self-attention layer is a linear transformation of the previous layer with non-linear weights.

Position-wise Feed-Forward Networks

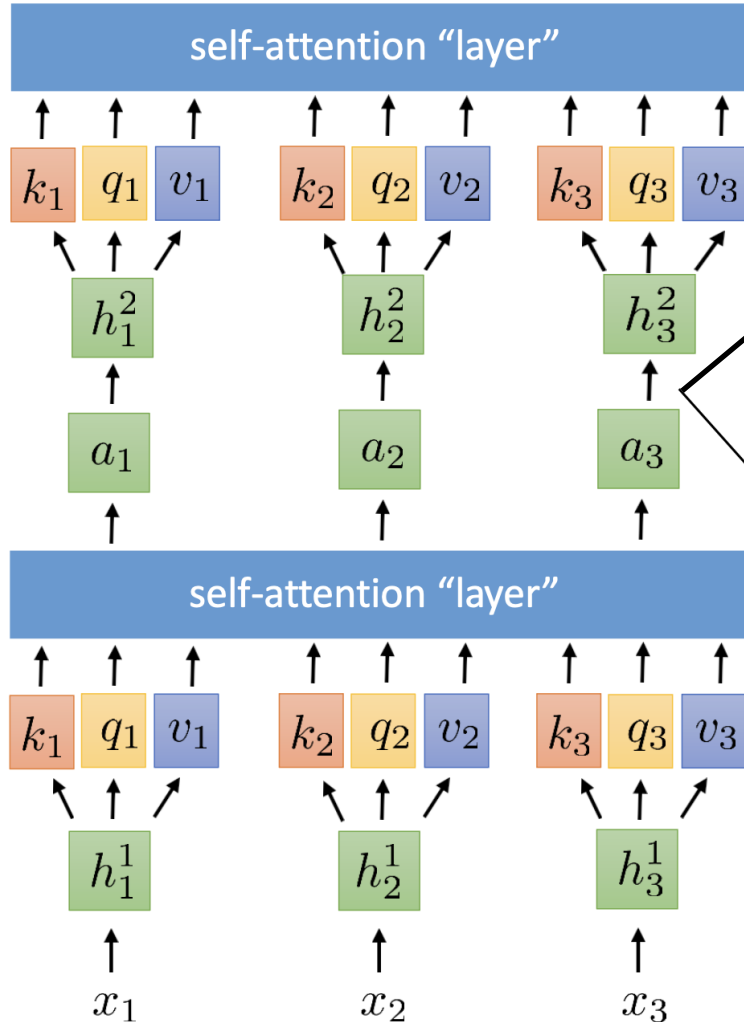


- **Solution:** Make the model more expressive is by alternating use of self-attention and non-linearity.
- Non-linearity is incorporated by means of a feed-forward network which consists of two linear transformations with a ReLU activation in between.

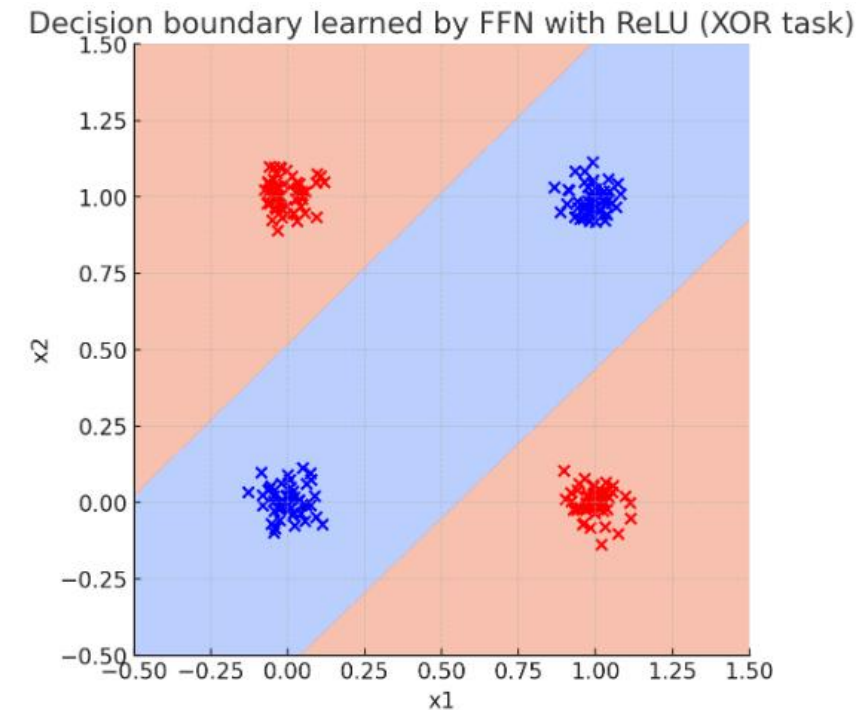
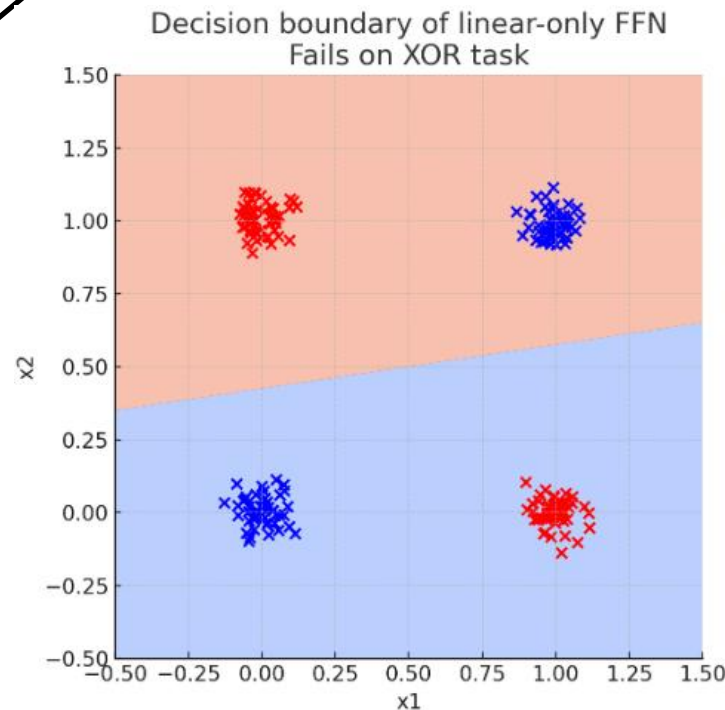
$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

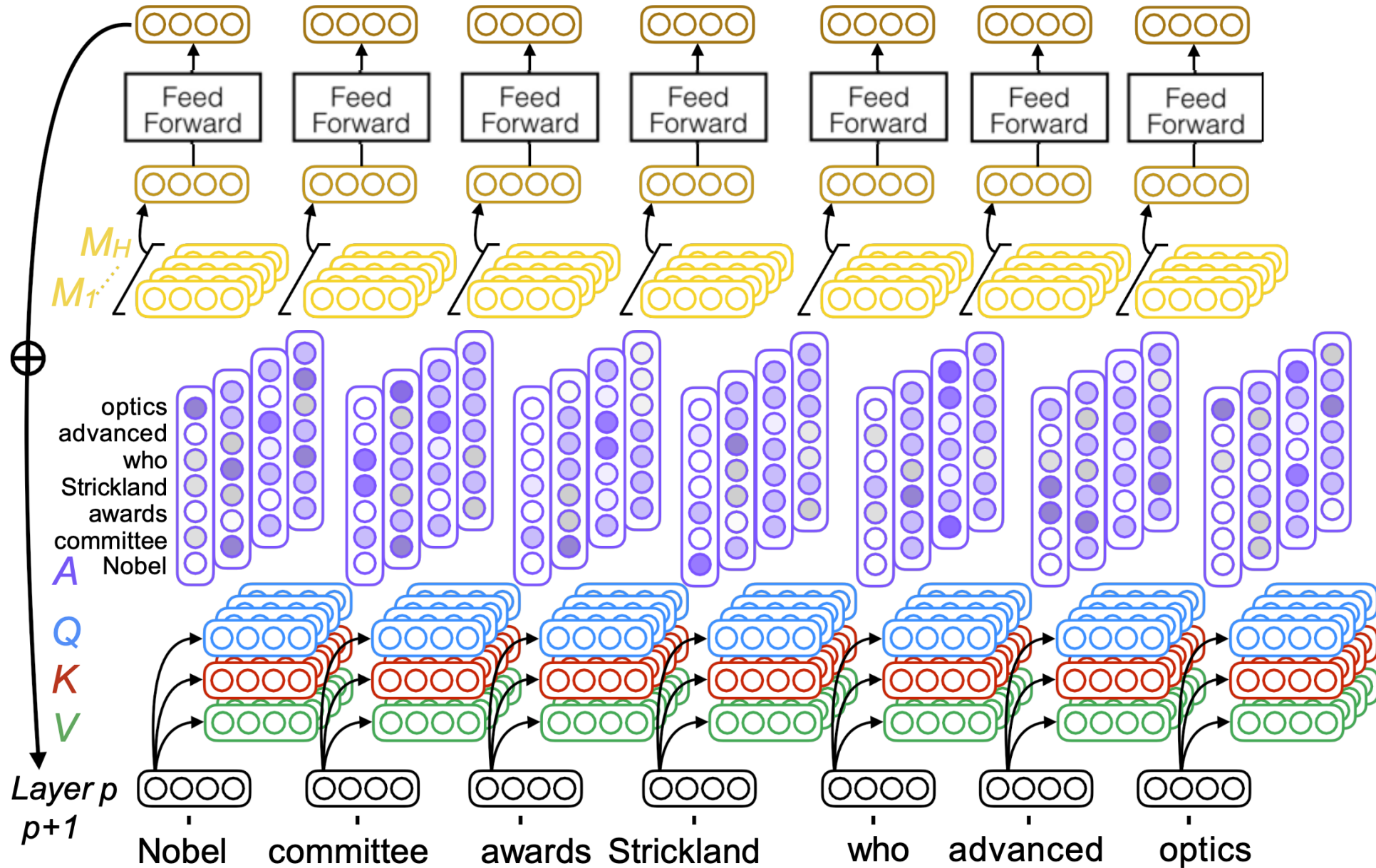
- The same non-linearity is utilized across various positions but they differ from layer to layer.

Position-wise Feed-Forward Networks



$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

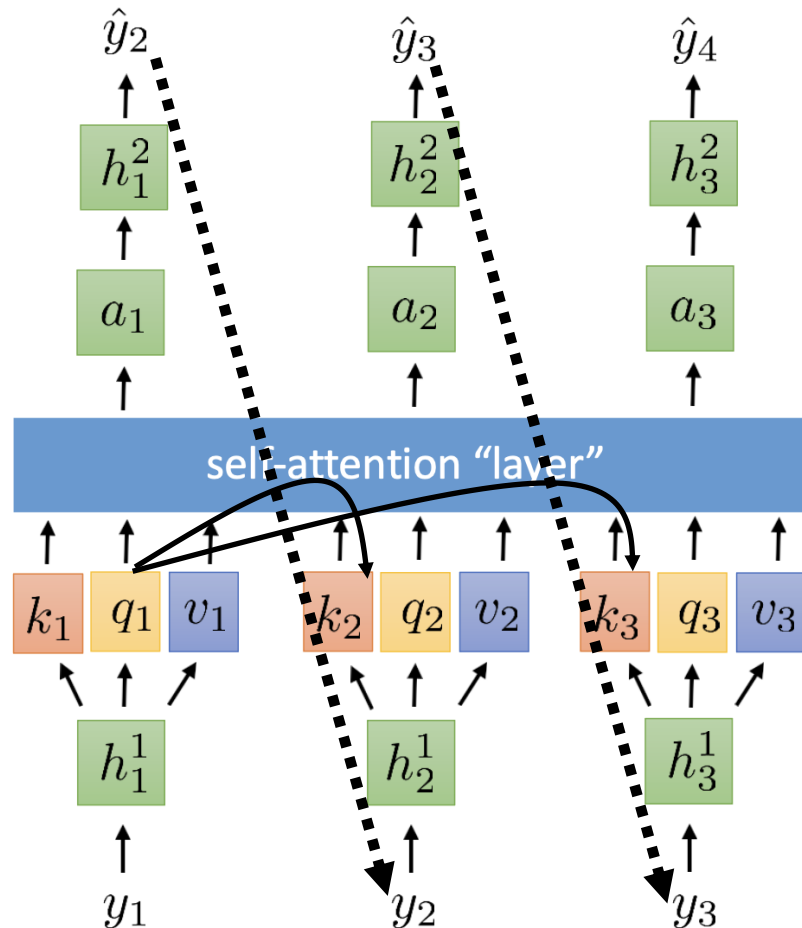




From Self-Attention to Transformers

- We will talk about a class of models for processing sequences that does not use recurrent connections but instead relies entirely on attention and will build up towards a class of models called **Transformers**.
- To address a few key limitations, we need to add certain elements:
 1. **Positional encoding**: Addresses lack of sequence information
 2. **Multi-headed attention**: Allows querying multiple positions at each layer
 3. **Adding nonlinearities**: So far, each successive layer is linear in the previous one
 4. **Masked decoding**: How to prevent attention lookups into the future?

Self-attention can see the future!



A **crude** self-attention "language model":

In practice, there would be several alternating self-attention layers and position-wise feedforward networks

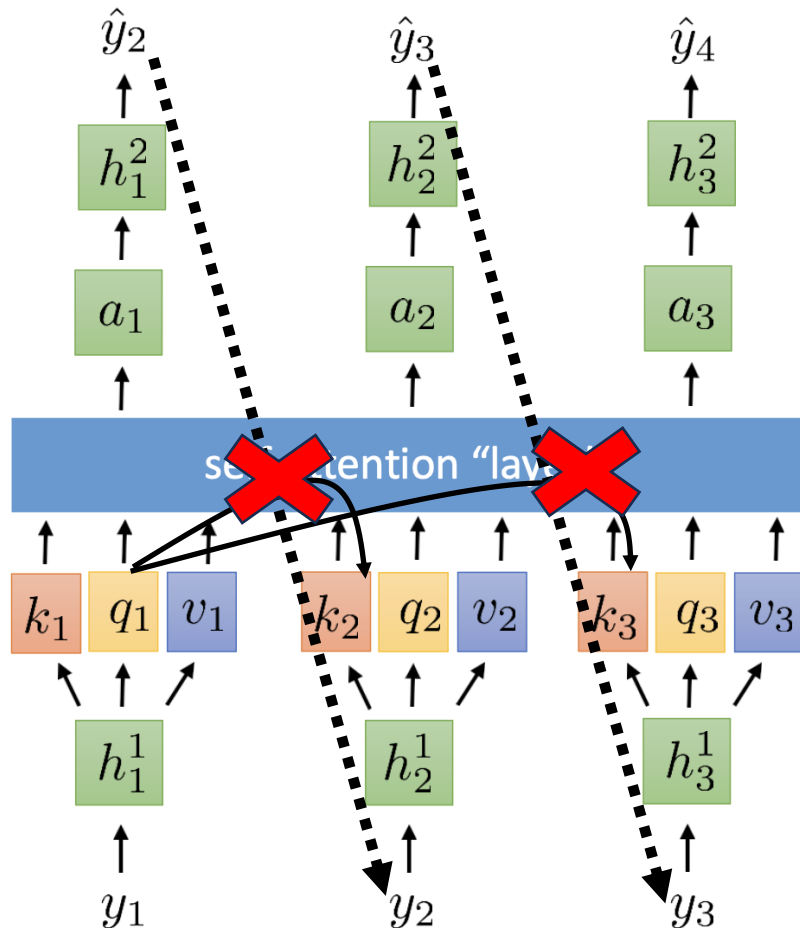
Big problem: self-attention at step 1 can look at the value at steps 2 & 3, which is based on the inputs at steps 2 & 3

At test time (when decoding), future tokens at steps 2&3 do not exist yet ...

...Therefore, step 1 must not attend to them.

Masked Attention

A **crude** self-attention “language model”:



At test time (when decoding), future tokens at steps 2&3 do not exist yet ...

...Therefore, step 1 must not attend to them.

Only allow self-attention into the past...
...but not into the future

Easy solution:

~~$$e_{l,t} = q_l \cdot k_t$$~~

$$e_{l,t} = \begin{cases} q_l \cdot k_t, & \text{if } l \geq t \\ -\infty, & \text{otherwise} \end{cases}$$

Masking the future in self-attention

We can look at these
(not greyed out) words

For decoding
these words

	[START] The chef who			
[START]		$-\infty$	$-\infty$	$-\infty$
The			$-\infty$	$-\infty$
chef				$-\infty$
who				

Necessities for a self-attention building block:

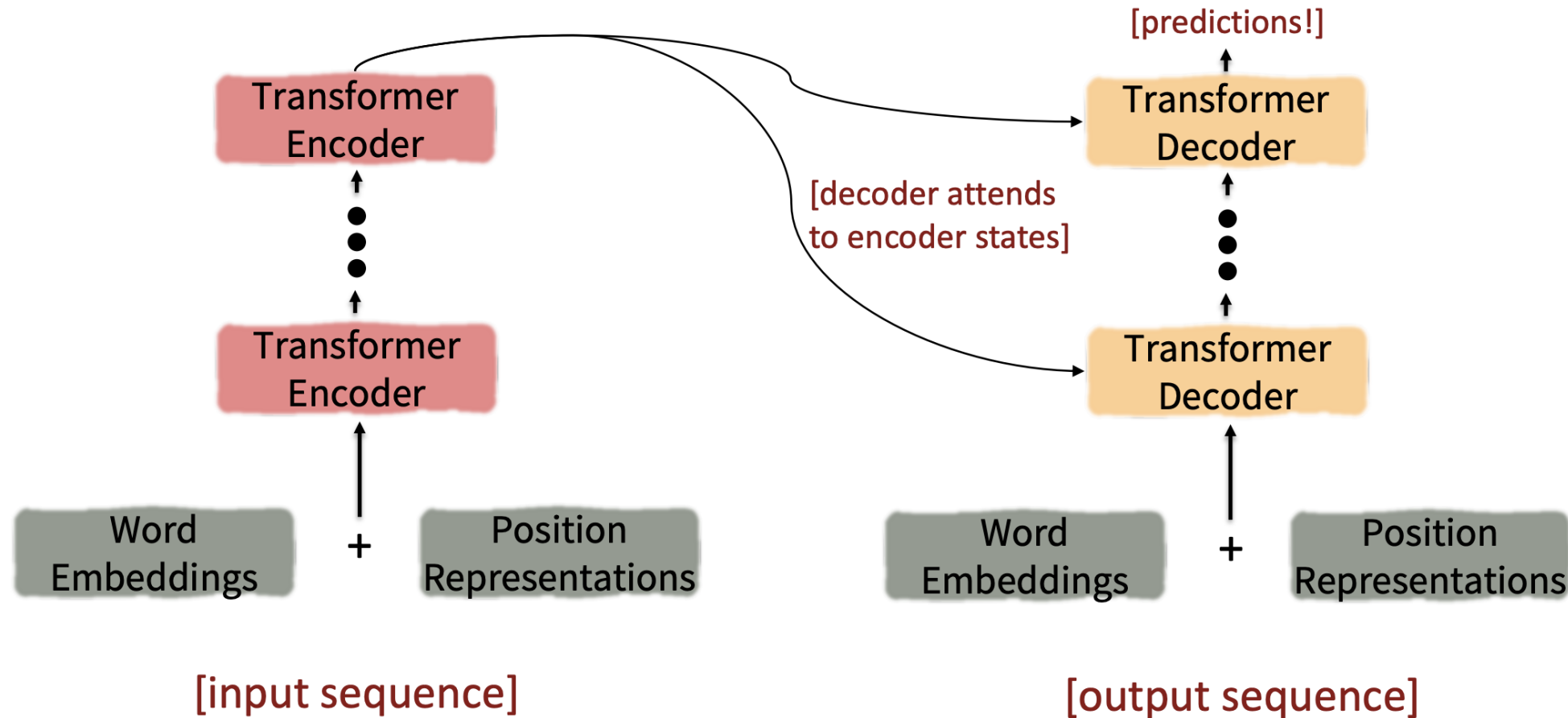
- **Self-attention:**
 - The basis of the method.
- **Position representations:**
 - Specify the sequence order, since self-attention is an unordered function of its inputs.
- **Nonlinearities:**
 - At the output of the self-attention block
 - Frequently implemented as a simple feed-forward network.
- **Masking:**
 - In order to parallelize operations while not looking at the future.
 - Keeps information about the future from “leaking” to the past
- That’s it! But this is not the **Transformer** model we’ve been hearing about.

Outline

- Self-Attention
- From Self-Attention to Transformers
- **Transformer Encoder and Decoder**
- **How to Fix**

The Transformer Encoder-Decoder (Vaswani et al., 2017)

First, let's look at the Transformer Encoder and Decoder Blocks at a high level



The Transformer Encoder-Decoder (Vaswani et al., 2017)

Next, let's look at the Transformer Encoder and Decoder Blocks

1. **Key-query-value attention:** How do we get the k , q , v vectors from a single word embedding?
2. **Multi-headed attention:** Attend to multiple places in a single layer!
3. **Tricks to help with training!**
 1. Residual connections
 2. Layer normalization
 3. Scaling the dot product
 4. These tricks **don't** improve what the model is able to do; they help improve the training process. Both of these types of modeling improvements are very important!

The Transformer Encoder: Key-Query-Value Attention

- We saw that self-attention is when keys, queries, and values come from the same source. The Transformer does this in a particular way:
 - Let $x_1, \dots, x_n$ be input vectors to the Transformer encoder; $x_i \in \mathbb{R}^d$
- Then keys, queries, values are:
 - $k_i = Kx_i$, where $K \in \mathbb{R}^{d \times d}$ is the key matrix.
 - $q_i = Qx_i$, where $Q \in \mathbb{R}^{d \times d}$ is the query matrix.
 - $v_i = Vx_i$, where $V \in \mathbb{R}^{d \times d}$ is the value matrix.
- These matrices allow different aspects of the x vectors to be used/emphasized in each of the three roles.

The Transformer Encoder: Key-Query-Value Attention

- Let's look at how key-query-value attention is computed, in matrices.
 - Let $X = [x_1; \dots; x_n] \in \mathbb{R}^{n \times d}$ be the concatenation of input vectors.
 - First, note that $XK \in \mathbb{R}^{n \times d}$, $XQ \in \mathbb{R}^{n \times d}$, $XV \in \mathbb{R}^{n \times d}$.
 - The output is defined as $\text{output} = \text{softmax}(XQ \cdot (XK)^\top) \cdot XV$

First, take the query-key dot products in one matrix multiplication: $XQ(XK)^\top$

$$\begin{array}{c} \text{pink box} \\ XQ \end{array} \begin{array}{c} \text{pink box} \\ K^\top X^\top \end{array} = \begin{array}{c} \text{pink box} \\ XQK^\top X^\top \end{array} \in \mathbb{R}^{n \times n}$$

All pairs of attention scores!

Next, softmax, and compute the weighted average with another matrix multiplication.

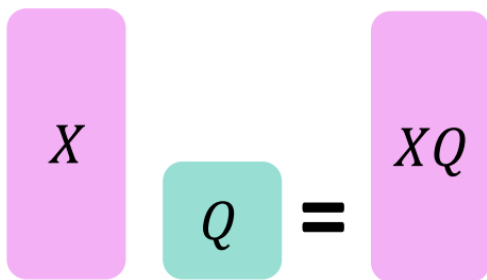
$$\text{softmax} \left(\begin{array}{c} \text{pink box} \\ XQK^\top X^\top \end{array} \right) \begin{array}{c} \text{pink box} \\ XV \end{array} = \begin{array}{c} \text{pink box} \\ \text{output} \end{array} \in \mathbb{R}^{n \times d}$$

The Transformer Encoder: Multi-head attention

- Look in multiple places in the sentence at once?
 - For word i , self-attention “looks” where $x_i^\top Q^\top K x_j$ is high, but maybe we want to focus on different j for different reasons?
- Define **multiple attention “heads”** through multiple Q, K, V matrices
- Let parameters $Q_l, K_l, V_l \in \mathbb{R}^{d \times d/h}$, where h is the number of attention heads, and l ranges from 1 to h .

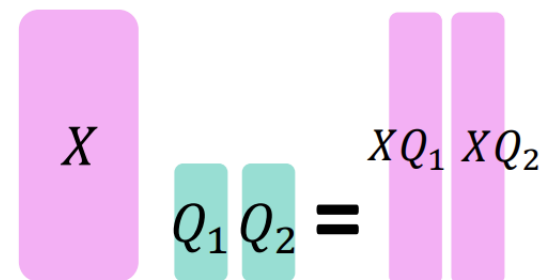
Single-head attention

(just the query matrix)



Multi-head attention

(just two heads here)



Same amount of
computation as
single-head self-
attention!

The Transformer Encoder: Residual connections [He et al., 2016]

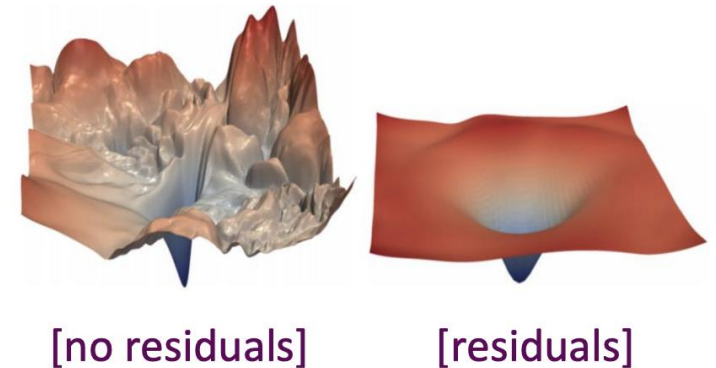
- Residual connections are a trick to help models train better.
 - Instead of $X^{(i)} = \text{Layer}(X^{(i-1)})$ (where i represents the layer)



- Let $X^{(i)} = X^{(i-1)} + \text{Layer}(X^{(i-1)})$ (so we only have to learn “the residual” from the previous layer)



- Residual connections are thought to make the loss landscape considerably smoother (thus easier training!)



[Loss landscape visualization,
[Li et al., 2018](#), on a ResNet]

The Transformer Encoder: Layer normalization (Ba et al., 2016)

- **Layer normalization** is a trick to help stabilize and improve optimization.
- Idea: Normalize each token's hidden vector to zero mean and unit deviation across its hidden dimensions, controlling its scale while retaining learned flexibility.
- Let $x \in \mathbb{R}^d$ be an individual (word) vector in the model.
- Let $\mu = \frac{1}{d} \sum_{j=1}^d x_j$; this is the mean; $\mu \in \mathbb{R}$.
- Let $\sigma = \sqrt{\frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2}$; this is the standard deviation; $\sigma \in \mathbb{R}$.
- Let $\gamma \in \mathbb{R}^d$ and $\beta \in \mathbb{R}^d$ be learned “gain” and “bias” parameters.
- Then layer normalization computes:

$$\text{output} = \frac{x - \mu}{\sigma + \epsilon} * \gamma + \beta$$

Normalize using scalar
mean and variance

Modulate by learned
elementwise gain and bias

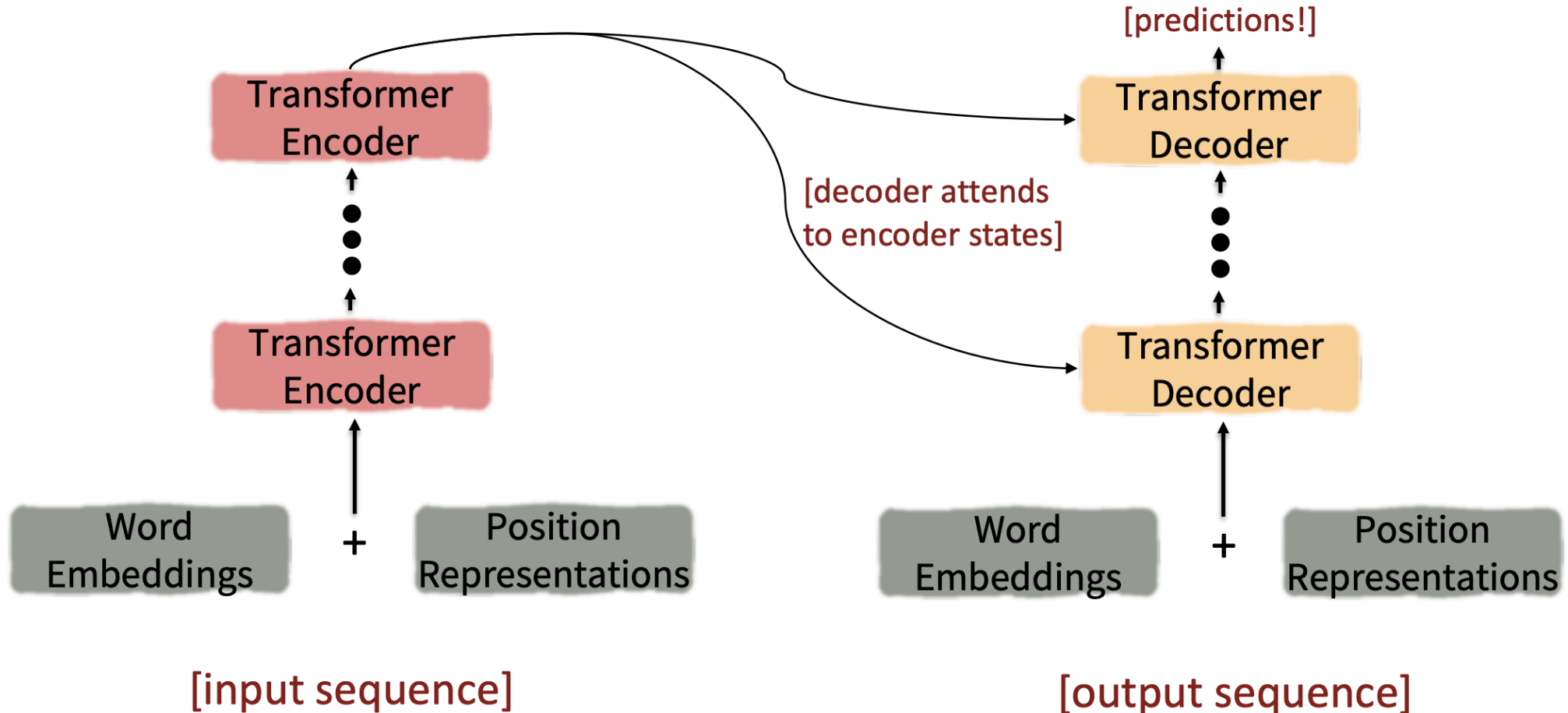
The Transformer Encoder: **Scaled Dot Product** (Vaswani et al., 2017)

- “**Scaled Dot Product**” attention is a final variation to aid in Transformer training.
- When dimensionality d becomes large, dot products between vectors tend to become large.
 - **Large dot products make softmax nearly one-hot; near 0 or 1, softmax derivatives are small.**
- We divide the attention scores by $\sqrt{d/h}$, to stop the scores from becoming large just as a function of d/h (The dimensionality divided by the number of heads.)

$$\text{output}_l = \text{softmax}\left(\frac{XQ_lK_l^\top X^\top}{\sqrt{d/h}}\right) * XV_l$$

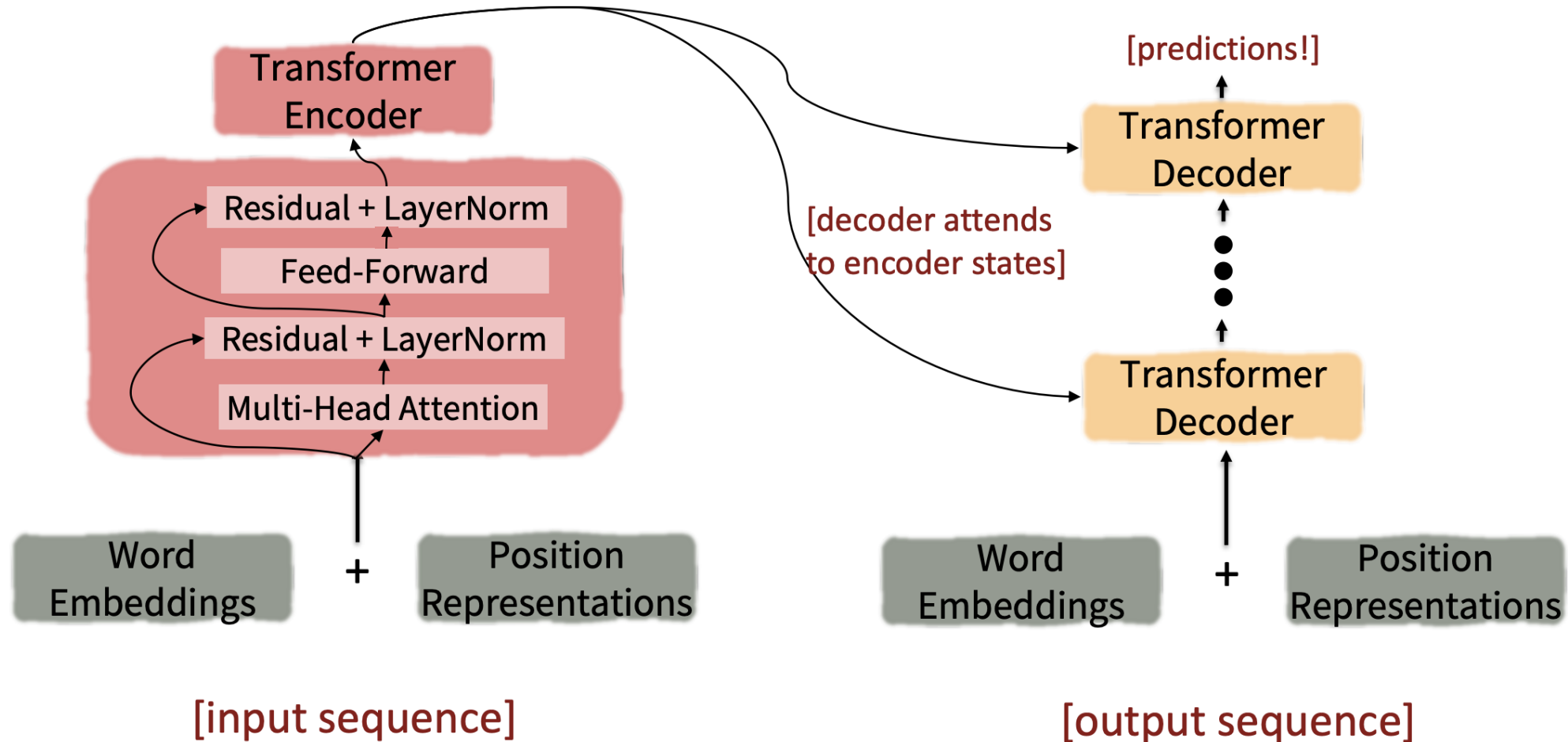
The Transformer Encoder-Decoder (Vaswani et al., 2017)

Looking back at the whole model, zooming in on an Encoder block:



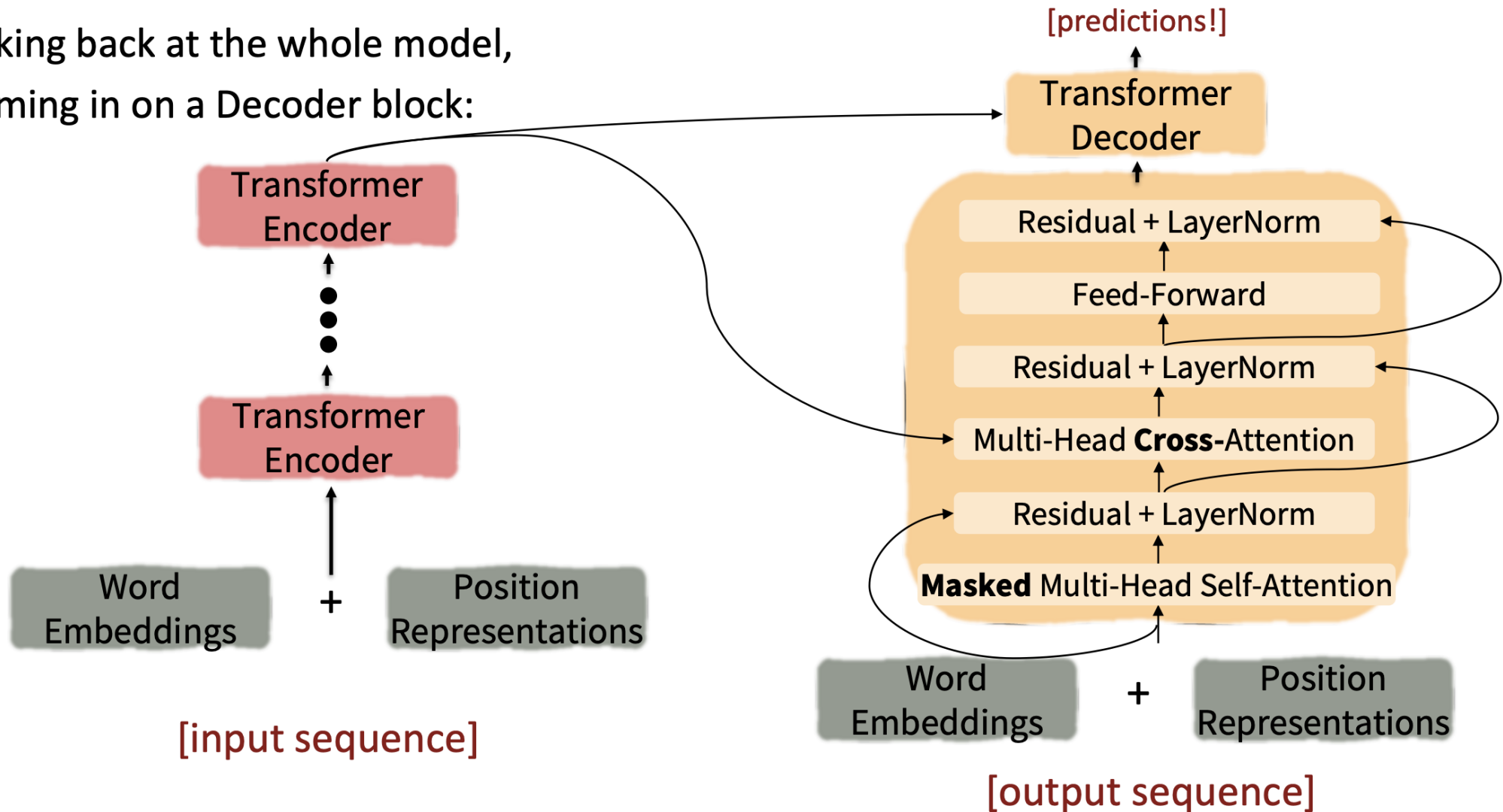
The Transformer Encoder-Decoder (Vaswani et al., 2017)

Looking back at the whole model, zooming in on an Encoder block:



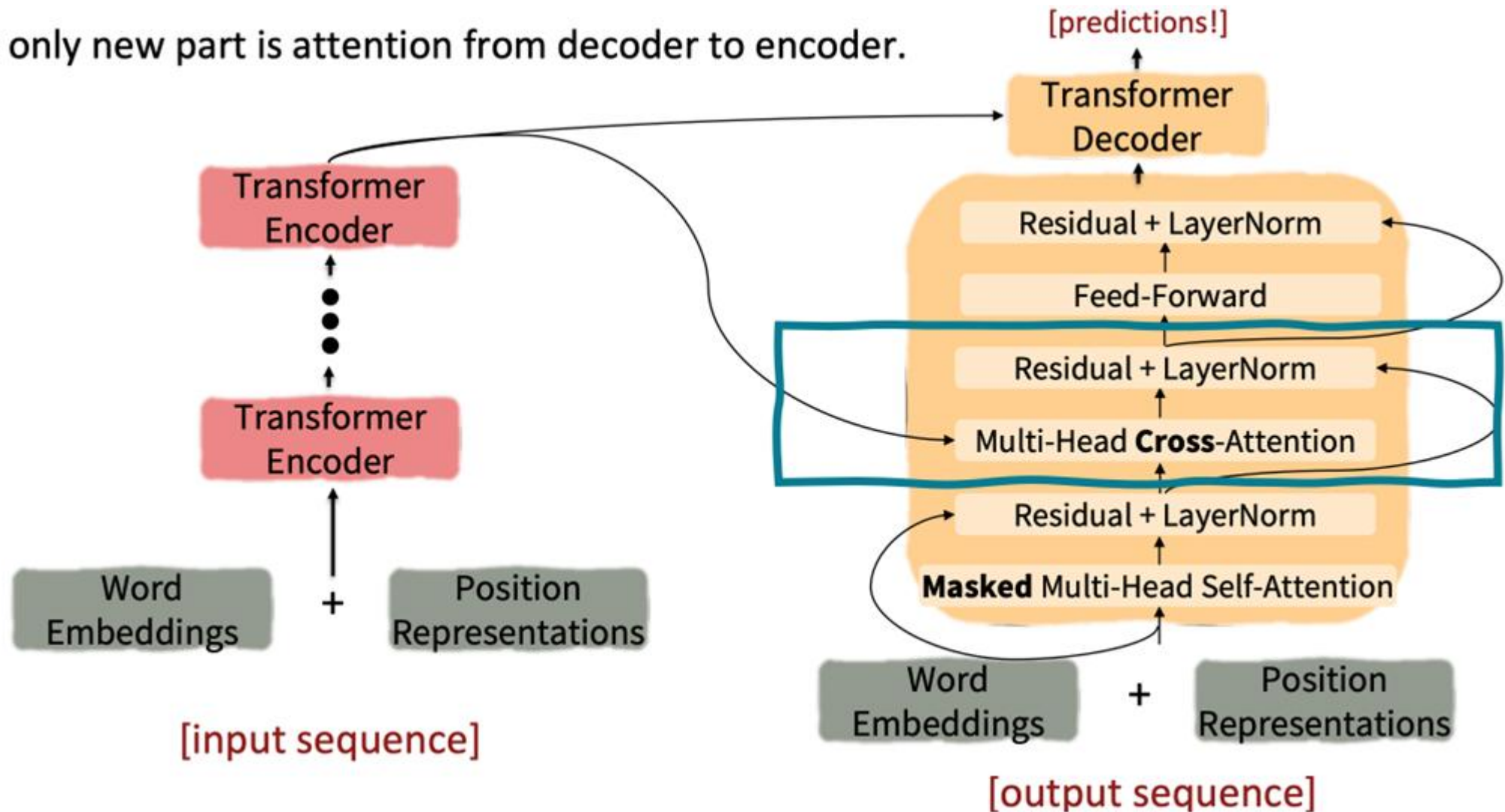
The Transformer Encoder-Decoder (Vaswani et al., 2017)

Looking back at the whole model,
zooming in on a Decoder block:



The Transformer Encoder-Decoder (Vaswani et al., 2017)

The only new part is attention from decoder to encoder.



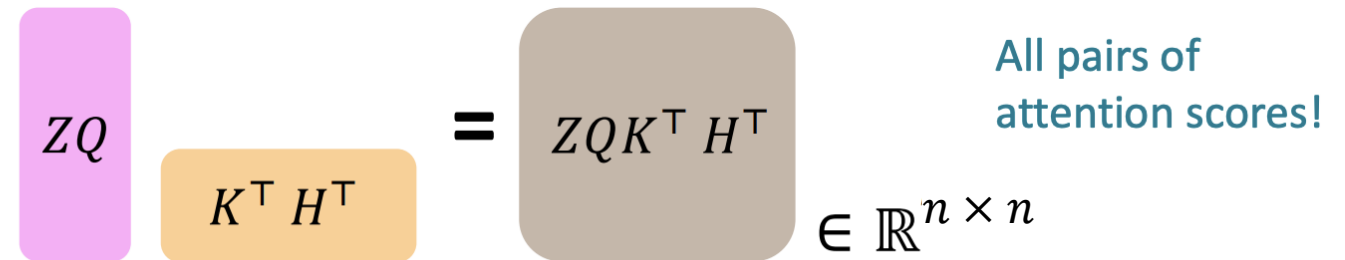
The Transformer Decoder: **Cross-attention** (details)

- We saw that self-attention is when keys, queries, and values come from the same source.
- Let $h_1, \dots, h_n$ be **output** vectors **from** the Transformer **encoder**; $x_i \in \mathbb{R}^d$
- Let $z_1, \dots, z_n$ be input vectors from the Transformer **decoder**, $z_i \in \mathbb{R}^d$
- Then keys and values are drawn from the encoder (like a memory):
 - $k_i = Kh_i, v_i = Vh_i$.
- And the queries are drawn from the **decoder**, $q_i = Qz_i$.

The Transformer Decoder: Cross-Attention (details)

- Let's look at how cross-attention is computed, in matrices.
 - Let $H = [h_1; \dots; h_n] \in \mathbb{R}^{n \times d}$ be the concatenation of encoder vectors.
 - Let $Z = [z_1; \dots; z_n] \in \mathbb{R}^{n \times d}$ be the concatenation of decoder vectors.
 - The output is defined as $\text{output} = \text{softmax}(ZQ(HK)^\top) \times HV$

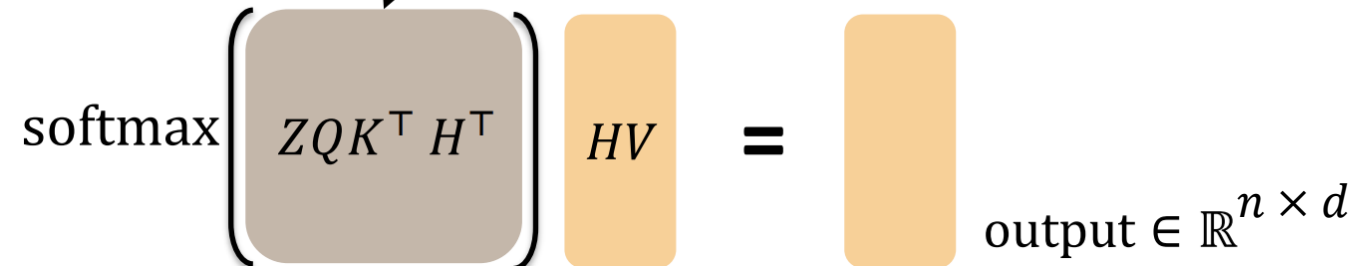
First, take the query-key dot products in one matrix multiplication: $ZQ(HK)^\top$



$$ZQ \quad K^\top H^\top = ZQK^\top H^\top \in \mathbb{R}^{n \times n}$$

All pairs of attention scores!

Next, softmax, and compute the weighted average with another matrix multiplication.



$$\text{softmax} \left(ZQK^\top H^\top \right) HV = \text{output} \in \mathbb{R}^{n \times d}$$

Outline

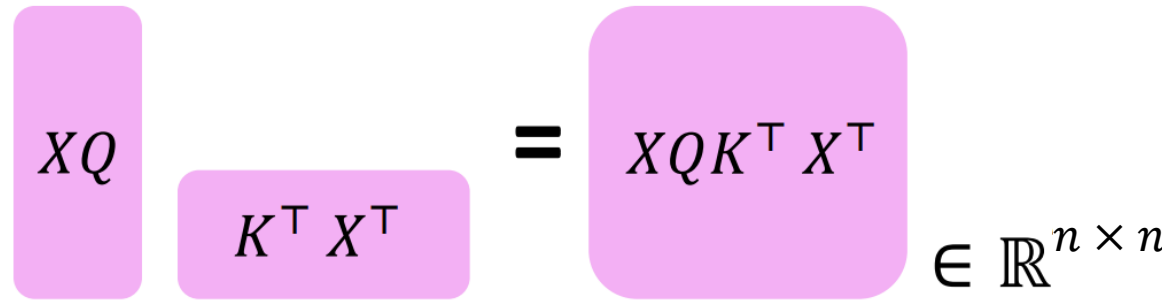
- Self-Attention
- From Self-Attention to Transformers
- Transformer Encoder and Decoder
- **How to Fix**

What would we like to fix about the Transformer?

- **Quadratic compute in self-attention (today):**
 - Computing all pairs of interactions means our computation grows **quadratically** with the sequence length!
 - For recurrent models, it only grew linearly!
- **Position representations (your self-study):**
 - Are simple absolute indices the best we can do to represent position?
 - Relative linear position attention ([Shaw et al., 2018](#))
 - Dependency syntax-based position ([Wang et al., 2019](#))

Quadratic computation as a function of sequence length

- One of the benefits of self-attention over recurrence was that it's highly parallelizable.
- However, its total number of operations grows as $O(n^2 d)$, where n is the sequence length, and d is the dimensionality.


$$XQ \quad K^T X^T = XQK^T X^T \in \mathbb{R}^{n \times n}$$

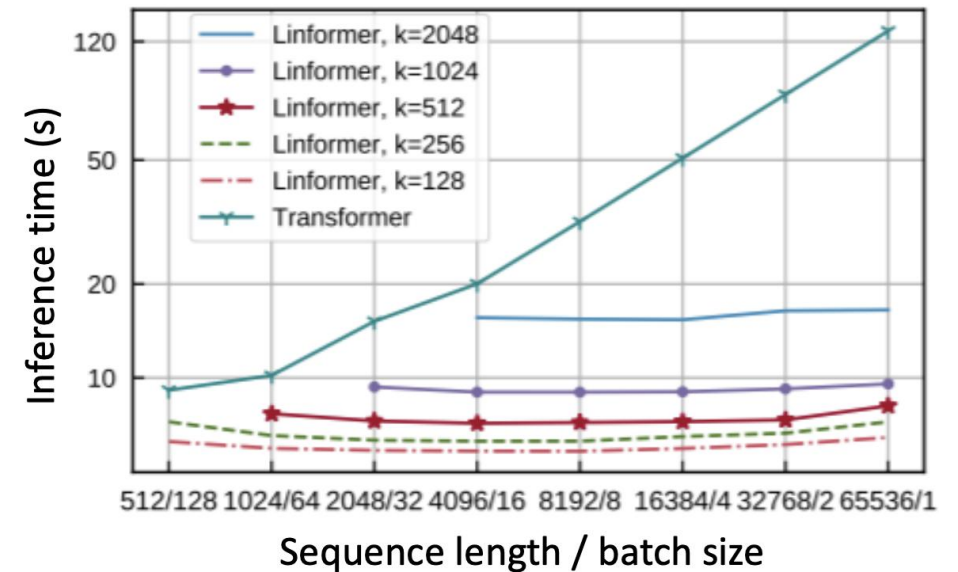
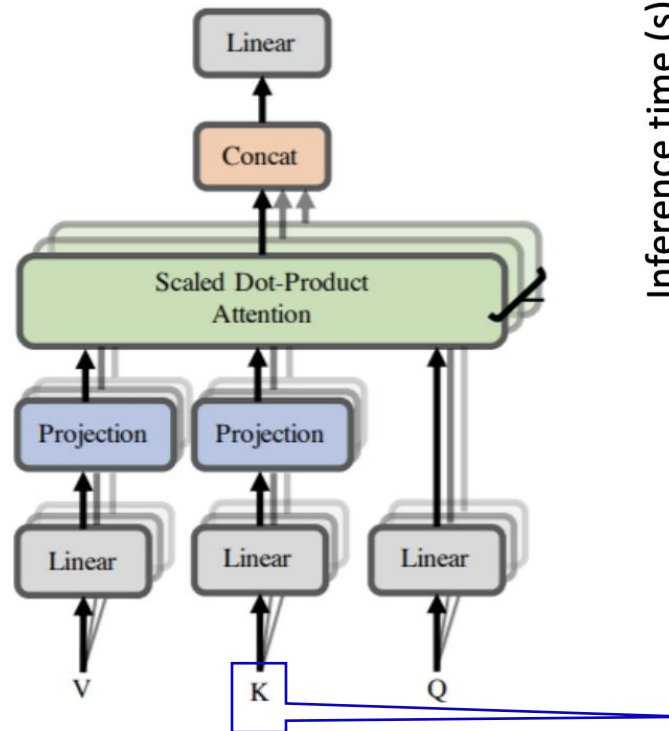
Need to compute all pairs of interactions!
 $O(n^2 d)$

- Think of d as around **1,000**.
- So, for a single (shortish) sentence, $n \leq 30$; $n^2 \leq$ **900**.
- In practice, we set a bound like $n = 512$.
- But what if we'd like $n \geq$ **10,000**? For example, to work on long documents?

Recent work on improving on quadratic self-attention cost

- Considerable recent work has gone into the question, *Can we build models like Transformers without paying the $O(n^2)$ all-pairs self-attention cost?*
- For example, **Linformer** ([Wang et al., 2020](#))

Key idea: keeps all n queries, but compresses the n key/value positions into $k \ll n$ learned summaries, reducing the number of attention comparisons.



Linformer

$$\begin{matrix} k \times n \\ \times \\ n \times d_m \\ \times \\ d_m \times d_k \\ = \\ k \times d_k \end{matrix}$$

The diagram shows the matrix multiplication for Linformer. It starts with a $k \times n$ matrix, which is multiplied by a $n \times d_m$ matrix. The result is then multiplied by a $d_m \times d_k$ matrix, resulting in a $k \times d_k$ matrix. The $n \times d_m$ matrix is labeled with a red 'X' and 'K', indicating that this step is not performed in the Linformer model.

Linformer (Wang et al., 2020)

The main idea of our proposed linear self-attention (Figure 2) is to add two linear projection matrices $E_i, F_i \in \mathbb{R}^{n \times k}$ when computing key and value. We first project the original $(n \times d)$ -dimensional key and value layers KW_i^K and VW_i^V into $(k \times d)$ -dimensional projected key and value layers. We then compute an $(n \times k)$ -dimensional context mapping matrix $\bar{P}$ using scaled dot-product attention.

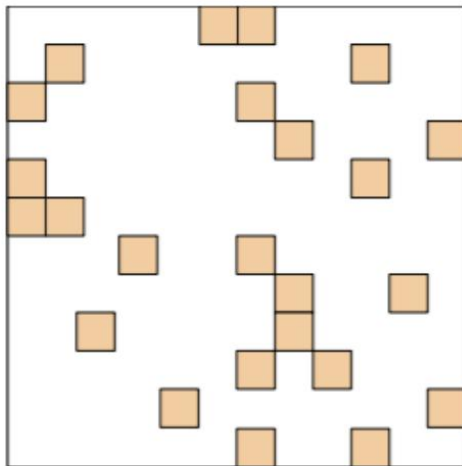
$$\begin{aligned} \overline{\text{head}}_i &= \text{Attention}(QW_i^Q, E_iKW_i^K, F_iVW_i^V) \\ &= \underbrace{\text{softmax} \left(\frac{QW_i^Q (E_iKW_i^K)^T}{\sqrt{d_k}} \right)}_{\bar{P}: n \times k} \cdot \underbrace{F_iVW_i^V}_{k \times d}, \end{aligned} \quad (7)$$

Finally, we compute context embeddings for each head_i using $\bar{P} \cdot (F_iVW_i^V)$. Note the above operations only require $O(nk)$ time and space complexity. Thus, if we can choose a very small projected dimension k , such that $k \ll n$, then we can significantly reduce the memory and space

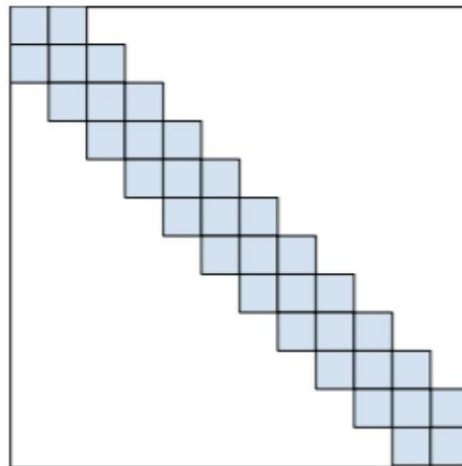
Recent work on improving on quadratic self-attention cost

- Considerable recent work has gone into the question, *Can we build models like Transformers without paying the $O(n^2)$ all-pairs self-attention cost?*
- For example, **BigBird** ([Zaheer et al., 2021](#))

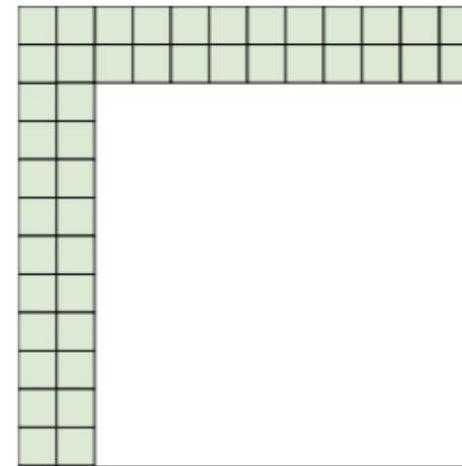
Key idea: replace all-pairs interactions with a family of other interactions, like **local windows**, **looking at everything**, and **random interactions**.



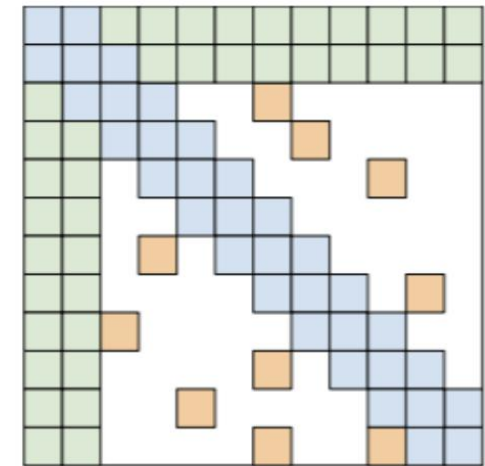
(a) Random attention



(b) Window attention



(c) Global Attention



(d) BIGBIRD

Big Bird

BigBird attention

- Combines **window, random, and global**.
- Local windows ensure nearby context.
- Random links ensure connectivity across distant positions.
- Global tokens ensure every token can indirectly access the full sequence.
- Overall complexity: $O(nw)$ (w is window size linear in sequence length, not quadratic!)

Theorem 3. *There exists a sparse attention mechanism using $O(n)$ inner products such that the resulting class of Transformer Networks using this sparse attention mechanism is Turing Complete.*

References

- Vaswani et al., “Attention is All You Need”. NIPS 2017
- Ba et al., “Layer Normalization”. arXiv:1607.06450
- Xu et al., “Understanding and Improving Lay Normalization”. NeurIPS 2019
- Wang et al., “Linformer: Self-Attention with Linear Complexity”. arXiv:2006.04768
- Zaheer et al., "Big Bird: Transformers for Longer Sequences". NeurIPS 2020